InterviewAce Test Strategy / Approach

AI-Powered Interview Practice Platform | Frontend, Backend, AI Scoring, Database, and Integration Testing

Document Type	Test Strategy / Approach
Project	InterviewAce - Job Interview Prep Tool
Primary Stack	React / TypeScript, FastAPI, PostgreSQL, SQLAlchemy, sentence-transformers, Ollama, Brevo SMTP, Docker / CI
Recommended Use	Project-level QA plan connecting requirements, test cases, traceability matrices, architecture diagrams, and implementation behavior

1. Purpose

This test strategy defines how InterviewAce should be validated across the user interface, backend APIs, AI scoring workflow, database persistence, and supporting services. The goal is to prove that the platform supports the complete job-seeker interview practice journey: authentication, role and level selection, question delivery, answer submission, four-axis scoring, deterministic feedback, AI coach interaction, and progress/history retrieval.

This document is intentionally practical and implementation-aware. It avoids testing endpoints or administrative workflows that are not clearly supported by the current codebase, and it focuses on the behavior described by the requirements specification, architecture diagrams, component diagram, frontend/backend test documents, and the repository design.

2. Testing Objectives

- Verify that all job-seeker-facing workflows work end-to-end from browser interaction to database persistence.
- Confirm that FastAPI endpoints enforce authentication, input validation, and ownership rules consistently.
- Validate that answer submission produces a persisted answer, a persisted score, deterministic rubric feedback, and a usable frontend response.
- Ensure the frontend remains usable on desktop, tablet, and mobile widths with no horizontal overflow or clipped chart/UI content.
- Confirm that integrations with PostgreSQL, sentence-transformers, Ollama, Brevo SMTP, and CI/CD are tested through controlled mocks or environment-specific checks.
- Maintain traceability from requirements to test cases, especially for authentication, password reset, question delivery, scoring, feedback, history, responsiveness, and persistence.

3. Scope

Area	In Scope	Out of Scope / Avoid
Frontend	React rendering, role/level selection, question UI, answer validation, score dashboard, history view, AI coach UI, responsive behavior.	Testing backend algorithms directly from frontend tests.
Backend API	Auth/register/login/reset flows, JWT protection, question retrieval, /scoring/submit, /history, validation and error handling.	Invented endpoints such as /sessions/start, /progress/radar, /feedback/{id}, or admin CRUD unless implemented in code.
AI / ML Scoring	sentence-transformers scoring pipeline, four score dimensions, overall score, deterministic numeric scoring, rubric-gap feedback.	Bit-for-bit validation of generative Ollama text, because LLM wording can vary.
Database	PostgreSQL persistence of users, questions, answers, scores,	Manual database edits as part of normal tests.

Area	In Scope	Out of Scope / Avoid
	feedback/history/chat records, migrations, restart persistence.	
External Services	Brevo email and Ollama integration through mocks, local test services, or controlled integration tests.	Sending real emails or depending on live external availability in every CI run.

4. Test Levels and Approach

Level	Primary Focus	Recommended Tools	Pass Criteria
Unit Tests	Pure functions, validators, scoring helpers, feedback rules, auth utilities, frontend components.	pytest, React Testing Library, Vitest/Jest	Fast, isolated, deterministic, no network/database dependency unless mocked.
API Integration Tests	FastAPI route behavior, JWT enforcement, validation errors, database reads/writes.	pytest + httpx AsyncClient, test PostgreSQL fixtures	Correct status codes, response schemas, ownership checks, persisted records.
Frontend Integration Tests	Role selection, question navigation, answer form validation, loading/error states, score/history rendering.	React Testing Library, mocked API client	User-visible states match expectations and no console/runtime errors occur.
End-to-End Tests	Full happy path: login -> choose role/level -> answer -> score/feedback -> history.	Playwright or Cypress	Flow completes without manual intervention and works at desktop/tablet/mobile widths.
Performance Tests	Score response time, frontend load performance, cache effectiveness.	k6/Locust, Lighthouse	Targets match NFRs: score response p95 <= 3s under stated load; LCP <= 2.5s.
Security Tests	JWT, password hashing, unauthorized access, input validation.	pytest, API tests, dependency checks	Missing/expired/tampered JWT returns 401; bcrypt hashes are never plaintext.
Regression / CI Tests	Protect core flows before merge.	GitHub Actions	All automated tests, lint, and build pass before main branch merge.

5. Test Strategy by System Area

5.1 Frontend Strategy

Frontend tests should validate the browser-facing experience without duplicating backend algorithm tests. Mock API responses for most component tests, then use one or more end-to-end tests to verify the integrated browser-to-backend path.

- Validate role and level selection: SWE, Data Science, PM, Behavioral; Intern and New Grad if exposed by the UI.
- Validate answer input behavior: empty/whitespace answers blocked, minimum answer length aligned to requirements, submit button disabled during loading.
- Validate score rendering: Technical Depth, Clarity, Completeness, Structure, overall score, feedback tips, and accessible text summary.
- Validate AI coach UI behavior: panel opens after scoring, messages wrap correctly, loading state appears, and long responses do not overflow.
- Validate history UI: empty state, chronological records, filters where implemented, and no blank/crashed pages.
- Run responsive checks at 375 px, 768 px, and desktop widths. No horizontal scrollbar, clipped labels, or chart overflow should be allowed.

5.2 Backend API Strategy

Backend tests should be endpoint-accurate. Based on the current project documents and repository behavior, the safest backend test surface is authentication, question retrieval, scoring submission, history retrieval,

password reset, and database persistence. Avoid using undocumented endpoints unless they exist in the actual code.

Backend Area	Core Tests	Important Notes
Authentication	Register/login, invalid credentials, missing fields, JWT issuance, protected endpoint access.	Map to FR-01 and NFR-07.
Password Reset	Forgot-password request, token generation, valid token reset, expired/invalid token rejection, old password fails.	Map to FR-02. Mock Brevo SMTP in CI.
Questions	Retrieve role/level questions, seeded rubric data present, invalid filters handled gracefully.	Map to FR-03 and FR-04.
Scoring Submit	POST /scoring/submit with valid answer, missing/short/whitespace answer, missing JWT, invalid question ID.	Should persist answer and score and return score + feedback.
Feedback	Feedback generated from missing rubric concepts and returned with scoring/history data.	Treat feedback as part of scoring unless a separate endpoint exists.
History	GET /history requires JWT, returns only current user data, empty state for new user.	Avoid /progress endpoints unless implemented.
Database	Verify records are linked by user_id/question_id/answer_id and survive restart/migration rebuild.	Map to NFR-10 and NFR-12.

5.3 AI Scoring and Feedback Strategy

The scoring workflow should be tested at two levels: deterministic numeric scoring and user-facing feedback quality. Numeric scoring can be strictly compared across repeated runs, while generated AI coach wording should be checked structurally rather than bit-for-bit.

- Submit the same answer to the same question multiple times and assert that four-axis numeric scores are identical or within an explicitly defined tolerance.
- Assert score dimensions are present and within 0-100: technical_depth, clarity, completeness, structure, and overall score if returned.
- Assert feedback references missing rubric concepts and contains actionable suggestions.
- Mock Ollama for CI to return a stable coaching response; run a separate local integration test for live Ollama behavior.
- Verify graceful fallback: scoring should still succeed if the AI coach service is unavailable.

5.4 Data and Persistence Strategy

- Use a dedicated test PostgreSQL database or isolated Docker test container.
- Seed users, questions, rubrics, answers, and scores using fixtures instead of relying on production data.
- Run migration rebuild checks: drop database, apply Alembic migrations, then verify schema and seed loading.
- After answer submission, directly confirm answer and score records exist and are linked to the authenticated user.
- Restart the backend or test container and confirm persisted answer/score records remain available through /history.

6. Recommended Test Suite Structure

Suite	Recommended Files / Groups	Purpose
Frontend Unit/Integration	frontend/src/**/*.test.tsx	Component rendering, validation, loading states, charts, responsive edge cases.
Backend API	backend/tests/test_auth.py, test_questions.py, test_scoring.py, test_history.py	Endpoint behavior, response schemas, auth protection, database writes.

Suite	Recommended Files / Groups	Purpose
Backend Services	backend/tests/test_scoring_service.py, test_feedback_agent.py	Scoring, feedback, deterministic behavior, cache behavior.
Database	backend/tests/test_migrations.py, test_persistence.py	Migration rebuild, persisted answers/scores, restart safety.
E2E	e2e/interview_practice.spec.ts	Full job-seeker practice flow from browser to score/history.
CI Smoke	GitHub Actions workflow	Fast set of core tests on every push/PR.

7. Requirement-to-Test Alignment

Requirement Area	Primary Test Coverage	Recommended Priority
FR-01 Authentication	Auth API tests, frontend login/register tests, protected route checks.	Critical
FR-02 Password Reset	Reset token tests, mocked Brevo email tests.	High
FR-03/FR-04 Role, Level, and Question Delivery	Question API tests and frontend selection/navigation tests.	High
FR-05 Answer Submission	Frontend validation, backend /scoring/submit tests, persistence checks.	Critical
FR-06/FR-07 Scoring and Score Display	Scoring service tests, response schema tests, frontend score display tests.	Critical
FR-08 Feedback	Feedback generation tests and UI rendering tests.	High
FR-09 AI Coach	Mocked Ollama integration test and frontend coach panel test.	Medium/High
FR-10 History	/history API test and frontend history rendering test.	High
NFR-03/NFR-05 UI Performance/Responsiveness	Lighthouse and responsive viewport tests.	Medium
NFR-07/NFR-08 Security	JWT enforcement and bcrypt hash verification.	Critical
NFR-10/NFR-12 Database Reliability	Migration rebuild and persistence-after-restart tests.	High
NFR-13 Deterministic Scoring	Repeated same-answer scoring test.	High

8. Test Environment and Data

Use three environments: local development, CI test environment, and demo/staging environment. The CI environment should use deterministic mocks for external services and a disposable database. Demo/staging should verify that the complete stack works with real deployment configuration but should not depend on real email delivery or nondeterministic AI outputs for pass/fail criteria.

- Test users: regular job seeker, new user with empty history, and optionally admin only if admin APIs exist.
- Question fixtures: at least one question per role/level with complete rubric and ideal answer fields.
- Answer fixtures: valid strong answer, short answer, whitespace-only answer, partially correct answer, repeated deterministic answer.
- Mocked services: Brevo SMTP and Ollama in CI; live local Ollama can be tested manually or in optional integration runs.
- Database fixtures: isolated test DB recreated per test run or transaction-rolled back between tests.

9. Entry and Exit Criteria

Entry Criteria	Exit Criteria
Requirements and diagrams are stable enough to map tests.	All critical and high-priority automated tests pass.
Test environment can run frontend, backend, PostgreSQL, and mocked services.	No open critical defects; no high defects in core practice flow.
Seed data and test users are available.	Requirement coverage is documented in RTM.

Entry Criteria	Exit Criteria
CI workflow can execute tests and build.	Demo smoke test passes: login -> questions -> submit -> score/feedback -> history.

10. Defect Classification

Severity	Definition	Examples
Critical	Blocks core system use or creates security/data-integrity risk.	Login impossible, unauthorized history access, score not persisted, app crash on submit.
High	Major feature broken but workaround may exist.	Questions do not load for a role, feedback missing, password reset broken.
Medium	Noticeable functional/UI issue that does not block core flow.	History filter incorrect, loading state missing, chart label truncated.
Low	Minor visual/content issue.	Typos, spacing inconsistency, non-blocking warning.

11. Automation and CI Approach

1. Run backend unit and API tests on every pull request.
2. Run frontend unit/integration tests on every pull request.
3. Run linting and build checks for both frontend and backend.
4. Run a small smoke E2E test suite before demo or release.
5. Use mocks for Brevo and Ollama in CI to keep the pipeline stable and fast.
6. Generate coverage reports and treat authentication, scoring, history, and persistence as required coverage areas.

12. Key Risks and Mitigations

Risk	Impact	Mitigation
Endpoint drift between documents and code	Tests fail for routes that do not exist.	Base backend tests only on implemented routes and update RTM when APIs change.
Nondeterministic AI coach responses	Flaky tests.	Mock Ollama in CI; validate structure rather than exact wording for live AI outputs.
Large frontend charts or text overflow on mobile	Poor usability and failed NFR-05.	Responsive tests at 375/768/desktop widths and manual visual review before delivery.
Database migration mismatch	Deployment failure or data loss.	Run migration rebuild test and persistence-after-restart test.
External email service unavailable	Password reset tests become flaky.	Mock Brevo for automated tests; run live email test manually or in staging only.

13. Final Recommendation

The best test approach for InterviewAce is a hybrid strategy: automated unit and API tests for backend correctness, frontend integration tests for UI behavior, a small E2E smoke suite for the complete job-seeker flow, and targeted non-functional tests for responsiveness, security, performance, deterministic scoring, migrations, and persistence. This is more appropriate than a single diagram because the project spans UI, API, AI/ML, database, and external service behavior.

Highest priority should be given to: authentication, password reset, question retrieval, /scoring/submit, deterministic score output, feedback generation, /history, JWT protection, bcrypt hashing, mobile responsiveness, and database persistence. Admin question-bank CRUD, session APIs, progress/radar

endpoints, and standalone feedback endpoints should only appear in tests if they are actually implemented in the codebase.